INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI

AN AGENTIC FRAMEWORK USING SLIME MOLD INTELLIGENCE FOR TRANSPORTATION NETWORK OPTIMIZATION

JITENDRA – 24623114

NIMAL S – 24623120

ROSHAN PAUL PATRICK – 24623152

THIRD YEAR

MINI PROJECT REPORT

Submitted to the

SCHOOL OF ENGINEERING

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

in Partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY CSE

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

VELS INSTITUTE OF SCIENCE, TECHNOLOGY AND ADVANCED STUDIES (VISTAS)

CHENNAI – 600 117

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**AN AGENTIC FRAMEWORK USING SLIME MOLD INTELLIGENCE FOR TRANSPORTATION NETWORK OPTIMIZATION**", is the Bonafide work of **JITENDRA - 24623114, NIMAL S - 24623120, ROSHAN PAUL PATRICK - 24623152** who carried out the project under my supervision. Certified further, that to the best of my knowledge the work reported here does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE OF THE HOD
Dr. K. Ulaga Priya M.E., Ph.D
Associate Professor & Head
Department of AI&ML
School of Engineering
VISTAS
Chennai

SIGNATURE OF THE SUPERVISOR
Dr. R. Anandan M.E., Ph.D
Professor
Department of AI&ML
School of Engineering
VISTAS
Chennai

Submitted for the Project Work and Viva-Voce examination held on……

at Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai – 600117.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

We hereby declare that the mini project report entitled "**AN AGENTIC FRAMEWORK USING SLIME MOLD INTELLIGENCE FOR TRANSPORTATION NETWORK OPTIMIZATION**", submitted to Vels Institute of Science, Technology and Advanced Studies (VISTAS), Chennai, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence), is a record of original work carried out by us under the guidance of **Dr. R. Anandan**, Department of Artificial Intelligence and Machine Learning. The work has not been submitted, in part or in full, for the award of any other degree or diploma.

Place: Chennai

Date: 30.10.2026

Jitendra – 24623114

Nimal S – 24623120

Roshan Paul Patrick – 24623152

ACKNOWLEDGEMENT

It is certainly a great delight and proud privilege to acknowledge the help and support we received from the positive minds around us in making this venture a successful one. The infrastructural support with all kinds of lab facilities has been a motivating factor in the completion of this project work, all because of our FOUNDER & CHAIRMAN, Shri. Dr. ISHARI K. GANESH, and it is with great pleasure that we take this opportunity to thank him. We are grateful to our HEAD OF THE DEPARTMENT, Dr. K. Ulaga Priya, for her continuous support for the project, from initial advice and contacts in the early stages of conceptual inception, and through on-going advice and encouragement to this day.

We extend our warmest thanks and deep gratitude to our INTERNAL GUIDE, Dr. R. Anandan, Department of AI&ML, for his patient guidance, regular reviews and constructive suggestions at every stage, from the choice of the topic to the preparation of this report. We would like to express our greatest gratitude to our faculty members, Librarian, and Non-teaching staff members of the Computer Centre for their support during our project reviews.

We also acknowledge the authors of the research papers cited in this report, in particular the work of Tero, Nakagaki, Jones, Tsompanas, Sirakoulis and Adamatzky, on which this project is built. Last but not the least, we wish to thank our parents and our friends for their individual support, patience and motivation, without whom we would be unable to complete this project.

JITENDRA SUNDESHA - 24623114

NIMAL S - 24623120

ROSHAN PAUL PATRICK - 24623152

CHAPTER NO.	TABLE OF CONTENT	PAGE NO.
	ABSTRACT	**i**
	LIST OF FIGURES	**ii**
	ABBREVIATIONS	**iii**
1	**INTRODUCTION**	**1**
	1.1 Overview of the project	1
	1.2 Scope and objective	4
2	**LITERATURE REVIEW**	**6**
	2.1 Literature Overview	6
	2.2 Literature Survey	12
3	**SYSTEM DESIGN**	**17**
	3.1 Existing System	17
	3.2 Proposed System	18
	3.3 System Architecture	21
	3.4 Hardware Requirements	23
	3.5 Software Requirements	24
4	**IMPLEMENTATION**	**26**
	4.1 Methodology	26
	4.2 Implementation	27
	4.2.1 Geospatial Data Collection Module	27
	4.2.2 Grid Mapping Module	28
	4.2.3 Simulation Initialization Module	28
	4.2.4 Slime Mold Agent Engine	29

	4.2.5 Network Extraction and Optimization Module	31
	4.2.6 Evaluation and Benchmarking Module	31
	4.3 Module Communication and Integration	33
5	**CONCLUSION AND FUTURE WORK**	**35**
	5.1 Conclusion	35
	5.2 Future Work	35
	5.3 Results	36
	APPENDIX A- CONFERENCE PAPER CERTIFICATES	**44**
	REFERENCES	**46**

ABSTRACT

Urban transportation and global logistics networks require massive infrastructure investments, and poor route planning leads to higher operational costs and longer travel times. Designing a transport network is a trade-off between three competing goals: the cost of building the links, the efficiency of travel, and the ability to keep working when a link fails. The plasmodium of the true slime mold Physarum polycephalum, a single-celled organism with no nervous system, spontaneously grows tube networks between food sources whose cost, efficiency and fault tolerance are comparable to those of the Tokyo rail system.

This project builds a computational pipeline based on this behaviour. Transport hubs are collected as latitude and longitude coordinates, converted into a two-dimensional grid, and placed on the grid as nutrient sources. A population of simple agents, following the multi-agent model of Jones, deposits and follows a diffusing chemoattractant. The population strengthens efficient routes and lets weaker and longer paths fade, and a stabilised network is then extracted and compared with the existing infrastructure.

The pipeline was applied to the Chennai Metro network The generated network was visually very close to the existing layout, differing at three locations. The comparison is qualitative, and the report defines an evaluation protocol based on normalised total length, mean shortest-path distance and single-link fault tolerance for the quantitative stage. The report also presents the design of a humanitarian logistics and emergency response extension, in which hazards and blocked links force the network to re-route and in which isolated settlements are connected by low-cost lifeline routes. This extension is proposed as a design and is identified as future work.

Keywords: Network Optimization, Logistics Routing, Sustainable Design, Bio-Inspired Computation, Slime Mold Algorithm, Physarum polycephalum, Multi-Agent Simulation, Emergency Response.

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO.
Figure 1	Proposed system architecture (6 modules in 3 layers)	21
Figure 2	Simulation in progress of the basic slime mold algorithm	37
Figure 3	Original Chennai Metro Network	38
Figure 4	Slime Mold Generated Chennai Metro Network	39

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
CA	Cellular Automaton
CSV	Comma-Separated Values
FT	Fault Tolerance
GIS	Geographic Information System
GPU	Graphics Processing Unit
GUI	Graphical User Interface
IDE	Integrated Development Environment
MD	Mean Shortest-Path Distance (Minimum Distance)
MST	Minimum Spanning Tree
ODE	Ordinary Differential Equation
OSM	OpenStreetMap
RA	Rotation Angle
RAM	Random Access Memory
RNG	Relative Neighbourhood Graph
SA	Sensor Angle
SO	Sensor Offset
TL	Total Length
VISTAS	Vels Institute of Science, Technology & Advanced Studies

CHAPTER 1. INTRODUCTION

1.1 OVERVIEW OF THE PROJECT

Introduction

A transport network is among the most capital-intensive pieces of infrastructure that a city or a country builds. Metro lines, freight corridors, highways and supply chains all connect a set of places, and the way they are connected decides how much material is needed for construction, how long journeys take, and how well the system survives when something goes wrong. Once a metro line or a rail corridor is built, its alignment is fixed for decades, so a poor planning decision is very expensive to correct.

Designing such a network is fundamentally a trade-off between three objectives that pull against each other:

- **Cost:** The total length of the links is a good proxy for the material and construction effort needed to build them.
- **Transport efficiency:** This is the distance that travellers or goods actually cover between pairs of places, measured by the average shortest-path distance in the network.
- **Fault tolerance:** This is the ability of the network to keep every place connected when one link fails or is closed.

Classical graph-theoretic tools handle these objectives badly when they are used alone. A minimum spanning tree (MST) is the cheapest way to connect a set of points, but every link in a tree is a bridge, so a single failure splits the network. A fully connected Delaunay triangulation is very robust, but in the Tokyo study discussed later it was about 4.6 times longer than the MST [1]. Real networks lie somewhere between these two extremes, and until recently there was no principled, nature-derived way to find that middle point.

Inspiration from the Slime Mold

Physarum polycephalum is a true slime mold. In its vegetative stage it is a single, large, multinucleate cell called a plasmodium, with no brain and no central control. When it is placed on a surface with food at several places, it first spreads widely to search the area, then withdraws from most of it and leaves a network of tubes that connects the food

sources [1]. Nakagaki and co-workers showed that it can find the shortest path through a maze [3]. Tero and co-workers showed that, when the food sources were laid out to match the positions of 36 cities around Tokyo, the network grown by the slime mold matched the real rail network in cost and efficiency, and approached it in fault tolerance [1].

The physical mechanism behind this behaviour is a positive feedback between the flow of fluid through a tube and the thickness of that tube. Tubes that carry more flow become thicker, which lowers their resistance and attracts even more flow. Tubes that carry little flow become thinner and eventually vanish [1], [4]. Because the total flow is fixed, tubes compete with one another, and only a small number of low-cost, high-flow paths survive, along with a few cross-links that add resilience. No part of the organism sees the whole network, yet the whole network is optimised.

This makes the slime mold an attractive template for decentralised, bottom-up network design, and the same idea can be written as an algorithm.

Problem Statement

Given the geographic positions of a set of transport hubs, the problem is to generate, by a decentralised bio-inspired process, a network that connects them with a good balance of cost, efficiency and fault tolerance, and to judge how closely that network resembles an existing, as-built network. A second, related problem is to make such a network adapt when links are blocked or when a hazard spreads, which is the situation in humanitarian logistics and emergency response.

System Overview

The project, titled An Agentic Framework Using Slime Mold Intelligence for Transportation Network Optimization, simulates the foraging behaviour of the slime mold on a computer. It integrates six main parts:

1. **Geospatial dataset:** A dataset of hub coordinates (latitude and longitude).
2. **Grid mapping:** A mapping step that converts the real-world map into a two-dimensional computational grid.
3. **Simulation initialisation:** A setup step that places the hubs on the grid as nutrient sources and starts the organism from a central node.

4. **Slime mold algorithm:** An agent engine in which many simple agents perform attractant diffusion, foraging and growth, and in which flux and conductance feedback strengthens good routes and removes weak ones.
5. **Network extraction:** An extraction step that reads the stabilised tubular network from the simulation.
6. **Evaluation and benchmarking:** An evaluation step that compares the simulated network with the actual as-built infrastructure.

The pipeline was applied to the Chennai Metro network as a first case study, and a basic version of the simulation is already working. The framework is described as agentic because the behaviour of the whole system emerges from a large number of autonomous agents, each following simple local rules, without any central planner. The design of a humanitarian logistics layer on top of this pipeline is also presented in this report, and it is identified clearly as a proposed extension.

Motivation

The motivation for the work comes from four observations:

- **Infrastructure permanence:** Infrastructure is expensive and permanent, so planners benefit from a quick, inexpensive way to ask whether a proposed or existing alignment is close to a nature-derived efficiency and robustness trade-off.
- **Lack of Indian validation:** The slime mold has been validated against real networks in Japan [1] and against motorway networks in five regions [6], but none of the studies reviewed for this project examines an Indian transport network.
- **Dynamic re-routing capability:** Physarum-based models have been shown to handle blocked links by removing the segment and solving again [7], and slime-mold-inspired models have been applied to crowd evacuation [11]. These results suggest a natural use of the approach in emergency response.
- **Disaster resilience:** Disasters such as floods and cyclones, which regularly affect coastal regions of India, make it important to have lifeline routes that can repair themselves and re-route around hazards.

1.2 SCOPE AND OBJECTIVE

Aim of the Project

The aim of the project is to design and build a slime-mold-inspired simulation pipeline that generates transport networks directly from the coordinates of hubs, to test it on the Chennai Metro, and to lay the foundation for its use in humanitarian logistics and emergency response.

Objectives

1. To study the biology of Physarum polycephalum and the main computational models of its network formation: the flux–conductance model, the multi-agent particle model and the cellular automaton model.
2. To build a pipeline that converts hub coordinates into a computational grid, places the hubs as nutrient sources and runs a multi-agent slime mold simulation.
3. To extract a stabilised network from the simulation output.
4. To apply the pipeline to the Chennai Metro station set and to compare the generated network with the existing one.
5. To define a quantitative evaluation protocol based on normalised total length, normalised mean shortest-path distance and single-link fault tolerance, following Tero et al. [1].
6. To present the design of an emergency-response extension that handles terrain barriers, spreading hazards and the connection of isolated settlements.

Scope of the Project

A. Completed Scope Reported in This Work

- A working simulation pipeline, from hub coordinates to an extracted network.
- A case study on the Green Line, the Blue Line (Mannadi to the Airport) and the Orange Line (Lighthouse to Poonamallee) of the Chennai Metro.
- A qualitative (visual) comparison between the generated and the existing network.
- The definition of the evaluation metrics and benchmarking procedure.

B. Proposed Design and Future Scope

- Obstacle and terrain handling through the image environment.
- Dynamic hazard-aware re-routing and self-repair for emergency evacuation corridors.

- Lifeline routes to isolated and remote settlements.
- Faster convergence of the simulation, and migration of the engine to the Rust language for large-scale parallel execution.
- Application to freight corridors, regional rapid transit and high-speed rail.

Significance of the Work

The approach needs only the coordinates of the hubs. It does not need a pre-defined graph, it runs without central control, and its raster representation can later carry terrain and barriers. If it is extended as planned, it can serve as a fast decision-support tool both for regional transit planning and for emergency management.

Organisation of the Report

Chapter 2 reviews the literature. Chapter 3 presents the existing and the proposed system, the architecture, the workflow, and the hardware and software requirements. Chapter 4 explains the methodology and the implementation of each module. Chapter 5 presents the conclusion, future work and experimental results, followed by the conference paper certificates in Appendix A and the references.

CHAPTER 2. LITERATURE REVIEW

2.1 LITERATURE OVERVIEW

The literature on slime-mold-inspired network design is reviewed here under seven themes. For each theme, the main findings are summarised and their relevance to this project is stated.

A. Biology of Physarum polycephalum

In its vegetative stage, Physarum polycephalum exists as a plasmodium, a single membrane-bound cell with many nuclei. The cytoplasm is organised into a contractile gel and a flowing fluid called the sol [5]. Starved on an agar plate with a single starting point, the plasmodium first spreads outwards with a broad, reticulated front that searches the maximum area. Once it meets food sources, it withdraws from most of the explored area and consolidates into a branching network of tubes that directly connects the food sources, occasionally adding extra junctions (Steiner points) that shorten the total network and a small number of redundant cross-links that improve transport efficiency and resilience [1].

Tero et al. describe two empirical rules that govern the changes in the tubular structure: tubes that are not connected to a food source tend to disappear, and when two or more tubes connect the same two food sources, the longer tubes tend to disappear [4]. The organism pumps fluid back and forth with rhythmic contractions that have a period of about two minutes, and tubes thicken in the direction in which this shuttle streaming persists. The result is the positive feedback between flux and tube thickness that is the core of all the computational models described below.

B. The Flux–Conductance Mathematical Model

Tero, Kobayashi and Nakagaki [4] formalised the feedback as a mathematical model on a graph. Each edge has a length and a conductivity. The flux through an edge follows a Poiseuille-type relation in which it is proportional to the conductivity and the pressure difference between the end nodes, divided by the length. Flow is conserved at every node, except at a single source and a single sink. The conductivity of each edge then evolves according to:

$$\mathbf{dD / dt = f(|Q|) - rD}$$

where D is the conductivity, Q is the flux, r is a decay rate and f is an increasing function with f(0) = 0. Edges that carry enough flux are reinforced, and edges that carry little flux decay towards zero and are effectively removed.

Two forms of f were studied: a power law f(Q) = Q^μ, and a sigmoidal form that saturates at large flux. The exponent μ has a decisive effect on the outcome. When μ > 1, the final network depends on the initial conductivities, and convergence to one dominant path is fast. When 0 < μ < 1, all competing paths survive whatever the initial conditions. When μ = 1, the shortest path always wins, whatever the initial state, but convergence is slower. The model evolves in two stages: first dead-end tubes are cut, and then a solution path is selected among competing paths [4]. This gives the designer a choice between a single canonical route and a set of alternative routes, depending on the planning question.

The same group then showed that this model can be used as a navigation tool. In a study with a map of the US interstate highway network, the shortest route from Seattle to Houston was found by the solver, and when a highway segment was reported blocked by an accident, the segment was removed, the source was reset to the current position, and the solver was run again to give a new optimum path [7]. This is the earliest indication in

the reviewed literature that the model can support re-routing, which is directly relevant to emergency response.

The limitation of this family of models is that they work on a pre-defined graph. They can rank or prune candidate connections, but they cannot by themselves propose new junctions in open space.

C. Validation Against Real Infrastructure

The best-known real-world validation is the Tokyo rail experiment of Tero et al. [1]. A plasmodium was grown from a start point representing Tokyo, in an arena bounded by the Pacific coastline, with 36 food sources marking the major cities of the region. The resulting network was compared with the real rail network using three measures, each normalised by the minimum spanning tree (MST) of the same points: total length (TL), the average minimum distance between pairs of nodes (MD) and fault tolerance (FT).

- **Total length ratio (TL / TL_MST):** 1.75 ± 0.30 for the slime mold, against about 1.8 for the real rail network. A Delaunay triangulation of the same points was about 4.6 times the MST.
- **Efficiency ratio (MD / MD_MST):** 0.85 ± 0.04 for the slime mold networks, effectively the same as the 0.85 of the rail network.
- **Fault tolerance:** The real network was slightly better: only 4% of single faults in the rail network would isolate a part of it, against 14 ± 4% for the illuminated slime mold networks and 20 ± 13% for the unconstrained ones.

The authors also proposed a mathematical model with sigmoidal feedback and random source and sink pairs. With suitable parameters, it produced networks with a benefit/cost ratio, defined as FT/TL_MST, of 0.7 against about 0.5 for the rail network and the slime mold, at an almost identical efficiency of 0.85.

Tsompanas, Sirakoulis and Adamatzky [6] extended this style of validation to motorway networks in five regions (the United Kingdom, Germany, Canada, Mexico and the Iberian Peninsula) using a cellular automaton model, described in Subsection E.

D. The Multi-Agent Particle Model

Jones [5] represents the plasmodium as a population of simple particle-like agents that move on a two-dimensional raster. Each agent has a position and a heading. At every step, it samples the concentration of a diffusing chemoattractant trail through three forward

sensors placed at a sensor offset (SO) from its body, at angles of plus and minus a sensor angle (SA) from its heading. It then rotates toward the strongest signal by a rotation angle (RA) and tries to move forward. A successful move deposits chemoattractant at the new location, whereas a blocked move deposits nothing and triggers a random change of direction. Because agents both deposit and follow the same substance, nearby agents reinforce each other's trails. The positive feedback of the slime mold therefore emerges from the population, rather than from an explicit equation.

Systematic variation of SA and RA gives a small number of pattern classes: reticular (highly connected), labyrinthine (dendritic) and island-like (disconnected). The sensor offset acts as a scale factor: small values of about 3 to 7 pixels give fine-grained networks, whereas larger values give coarser networks with thicker trunks [5].

When fixed attractant sources, analogous to cities, are projected on to the trail map as pre-patterning cues, the population condenses around them and then removes redundant links under an emergent tension effect, until the network approximates a minimum Steiner tree. A stronger projection weight tightens the network towards an MST, and a weaker weight preserves cross-links for redundancy. Jones also showed that a dense sheet of plasmodium can be shrunk on to a set of stimulus points by randomly removing agents, and that networks can repair themselves when part of the pattern is destroyed [5].

This model operates directly on a rasterised map, which is the natural form of a metro or freight corridor layout. For this reason it is adopted as the core engine of the present project.

E. The Cellular Automaton Model and Evacuation

Tsompanas et al. [6] describe a cellular automaton (CA) in which each cell of a grid holds a percentage of Physarum mass and of chemoattractant. Both diffuse over a Moore neighbourhood, and the mass is drawn toward neighbouring cells with a higher attractant concentration. Nutrient-source cells that accumulate enough mass become new starting points, and a tube is drawn by tracing the steepest ascent of the mass field back to an existing source. The model was checked against the networks that real plasmodia built on templates of five national motorway systems, and the following was reported:

- The graphs built by the model matched 91.3% of the graphs grown in laboratory experiments.

- The model reproduced the relative neighbourhood graph (RNG) completely for the United Kingdom and Germany, and with small omissions for Canada (14 of 15 edges), Mexico (17 of 20) and the Iberian Peninsula (24 of 26).
- The intersection with the real motorway network exceeded 50% of its edges, and redundant edges did not exceed 24% of the model's edges.
- Every run took no more than about 10 minutes on a commercial computer, against the 3 to 5 days that a laboratory plasmodium needs.

The authors attribute the remaining difference from real motorways to terrain, population and political and economic constraints that the model does not take into account [6].

Kalogeiton et al. [11] used a related CA model for crowd evacuation. The model includes food diffusion, the growth of the organism, the creation of tubes and the selection of the best tube for each person, so that each person moves toward the nearest exit. It was tested in virtual environments with corridors, U-shaped obstacles, multiple exits and multiple rooms, and in a real indoor environment, namely the first floor of a university building, with 61 and 200 persons. The velocity–density and flow–density diagrams that the model produced agreed with those published for real crowds, and real Physarum experiments in a scaled-down layout of the same floor supported the model [11]. In this model, walls are cells that cannot be occupied, and exits act as food sources. This is useful for the emergency-response direction of this project.

F. Acceleration and Optimisation Variants

A recurring limitation is the computation time. Graph-based Physarum solvers must solve a linear system for the pressure at every node on every iteration, and they may need hundreds of iterations.

Zhang et al. [8] introduced an 'energy' parameter into the basic model so that the energy supplied by the flux and the energy consumed by the tube length are balanced. On random networks of 15 to 2000 nodes, the improved algorithm needed fewer iterations and less time than the basic one, and it found the optimal path in all tests, as Dijkstra's algorithm does. It was slower than Dijkstra's algorithm, but it could identify more than one shortest path at once.

Gao et al. [9] reduced the time in two ways. They pruned the nodes and edges that become inactive and they stopped the solver early, once a small set of near-optimal paths carried most of the flow. On sparse random graphs, an average of 55.54% of the nodes

and 77.85% of the edges were inactive after only 26 iterations, whereas the classical solver needed on average 327.5 iterations to converge. The method was also tested on two real traffic networks of about 950 nodes.

Awad et al. [12] proposed a Physarum competition algorithm for discrete multi-objective optimisation on a hexagonal cellular automaton, in which competing organisms explore the search space and repel one another. It gave the best spread of solutions in a benchmark against NSGA-II, NSGA-III and SPEA2.

Liu et al. [10] developed the Physarum optimisation for the Steiner tree problem in networks, an important NP-hard problem in network design, and applied it to the minimal exposure problem in wireless sensor networks. These accelerations were designed for graph solvers. Their transfer to a grid-based agent model needs separate work.

G. Logistics and Supply Chain Applications

Physarum-inspired models have also been applied to logistics. Chu et al. [13] defined an effective distance for logistics networks that accounts for economies of scale and the cooperation of suppliers, and they solved the resulting cost-minimisation problem with a modified Physarum algorithm that keeps the flows of different origin–destination pairs separate. They report better computational performance than other methods such as particle swarm optimization. Cai et al. [14] proposed an artificial Physarum swarm for the robust design of logistics networks. They defined four robustness indicators (relative, betweenness, edge and closeness robustness) and tested the method on an example based on Mexico City, where the swarm improved the robustness of the network under disruptions.

In the wider field of biologically inspired networking, Nakano [2] reviews systems inspired by the nervous system, the immune system, insect colonies, genetic evolution and cellular development, and identifies four design principles that they share: large numbers of redundant components, local interactions that give rise to collective behaviour, a stochastic nature that supports exploration, and feedback-based control. All four are present in the slime mold algorithm.

2.2 LITERATURE SURVEY

Summary of the Reviewed Papers

Table 1: Summary of the reviewed papers

No.	Authors (Year)	Title / Topic	Method	Main finding	Limitation relevant to this project
1	Tero, Kobayashi, Nakagaki (2007) [4]	A mathematical model for adaptive transport network in path finding by true slime mold	Coupled ODEs on a fixed graph; power-law and sigmoidal feedback	The exponent μ decides whether one path or all paths survive; dead-end cutting then path selection	Works only on pre-defined graphs; cannot discover new junctions in open space
2	Tero, Kobayashi, Nakagaki (2006) [7]	Physarum solver: a method of road-network navigation	Flux–conductance solver on a road map	Shortest route on the US interstate network; re-routing after a blocked segment by resetting the source	Shown for path-finding between two points, not for multi-hub network design
3	Tero et al. (2010) [1]	Rules for biologically inspired adaptive network design	Laboratory Physarum on a Tokyo template; sigmoidal flux model	TL/TL_MST = 1.75 and MD/MD_MST = 0.85, comparable to the Tokyo rail network	Parameters tuned for the Tokyo topology; one real-world case
4	Jones (2010) [5]	Characteristics of pattern formation and evolution in approximations of Physarum transport networks	Multi-agent particle system on a raster map	Parameter classes (SA, RA, SO); Steiner-tree behaviour, network minimisation and self-repair	High computational burden; qualitative comparison only
5	Tsompanas, Sirakoulis, Adamatzky (2015) [6]	Evolving transport networks with cellular automata models inspired by slime mould	Cellular automaton with diffusing mass and attractant	91.3% match with laboratory graphs for five motorway networks; minutes instead of days	Largest case studied is small (21 German cities); terrain and politics not modelled

6	Kalogeiton et al. (2015) [11]	Cellular automaton model of crowd evacuation inspired by slime mould	CA with food diffusion, tube creation and exit selection	Realistic velocity–density and flow–density diagrams; tested on a real building floor	Micro-level indoor crowd dynamics; not a regional network
7	Zhang et al. (2014) [8]	An improved Physarum polycephalum algorithm for the shortest path problem	Energy parameter added to the flux–conductance model	Fewer iterations than the basic model on 15 to 2000 nodes; finds multiple shortest paths	Slower than Dijkstra's algorithm; graph-based
8	Gao et al. (2018) [9]	An accelerated Physarum solver for network optimization	Pruning of inactive nodes and early termination	About 55% of nodes and 78% of edges inactive after 26 iterations against 327.5 iterations to converge	Targets shortest-path graph solvers, not grid-based agent models
9	Awad et al. (2019) [12]	A Physarum-inspired competition algorithm for discrete multi-objective optimization	Competing Physarum on a hexagonal CA	Best spread against NSGA-II, NSGA-III and SPEA2 on the Deep Valley problem	Not a network-design method; short two-page report
10	Liu et al. [10]	Physarum optimization for the Steiner tree problem in networks	Physarum cellular computing model	Low-complexity, highly parallel Steiner tree algorithm applied to minimal exposure in sensor networks	Sensor-network context; no geographic transport data
11	Chu et al. (2021) [13]	A Physarum-inspired algorithm for logistics optimization: effective distance	Modified Physarum solver with effective distance	Cost reduction through supplier cooperation and economies of scale	Graph-based; numerical examples only
12	Cai et al. (2022) [14]	Design a robust logistics network with an artificial Physarum	Artificial Physarum swarm with four robustness indicators	Improved robustness under disruption on a	Graph-theoretic; no raster or terrain representation

		swarm algorithm		Mexico City example	
13	Nakano (2011) [2]	Biologically inspired network systems: a review	Survey of in silico and in vitro/vivo network systems	Four shared design principles: redundancy, local interaction, stochasticity, feedback	General review; does not address transport design

Comparison of the Three Main Model Families

Table 2: Comparison of the three main model families

Model	Representation	Strength	Weakness	Role in this project
Flux–conductance ODE [4]	Fixed graph	Analytically tractable; compact	Cannot create new junctions	Mathematical reference and baseline on small sub-graphs
Multi-agent particle [5]	Raster map with many agents	Works on raw maps; Steiner points; self-repair; tunable parameters	High computational cost	Core simulation engine
Cellular automaton [6]	Grid of mass and attractant fields	Fast; validated on five motorway networks	Fewer tunable behaviours than agents	Basis of a faster large-scale variant

Research Gaps

The review shows the following gaps, which this project addresses in turn:

- **No Indian case:** The networks studied in the reviewed literature are those of Japan, Europe, North America and Mexico. None of the reviewed studies addresses an Indian transport network.
- **Predefined graphs and static networks:** Prior work mostly evaluates models on predefined graphs or static networks in developed regions, and graph-based solvers cannot find new junctions in open space [4].
- **Terrain treated as a uniform plane:** Real transport networks are affected by water bodies, hills and other obstacles. The raster environment of [5] and the CA of [6] can encode these, but this has not been tested for Indian cities.

- **Biological mimicry over deployment:** Most studies aim to reproduce laboratory observations and give limited attention to practical infrastructure planning tools.
- **Dynamic logistics such as evacuation routing:** Evacuation is addressed at a micro scale inside buildings [11], and re-routing is shown on graphs [7], but there is no regional-scale simulation of self-repairing evacuation corridors.
- **Links to isolated communities:** The design of resilient, low-cost connections to geographically isolated settlements has not been addressed in the reviewed work.
- **Computation time:** Agent-based models are costly [5], while the available accelerations are for graph solvers [9].

Positioning of This Project

The project adopts the multi-agent model of Jones [5] as its core engine because the input to the problem, a set of points on a map, is naturally a raster. The flux–conductance model [4] is used as a mathematical reference, and the evaluation uses the three measures of Tero et al. [1] and the comparison strategy of Tsompanas et al. [6]. The project contributes a working pipeline from coordinates to network, a first Indian case study, an evaluation protocol, and a design for extending the pipeline to emergency response.

CHAPTER 3. SYSTEM DESIGN

3.1 EXISTING SYSTEM

Overview

In current practice, a transport network is planned by a combination of demand forecasts, land availability, cost estimates and political and economic priorities. Computational tools support this process in several ways:

- **Shortest-path algorithms:** Dijkstra's algorithm and the Bellman–Ford algorithm find the shortest route between nodes in a given weighted graph [8]. They need the graph to be defined beforehand and they give one route, and the computation time grows with the number of nodes [7].
- **Proximity and spanning graphs:** The minimum spanning tree gives the cheapest connection and the Delaunay triangulation gives the most connected one. Neither one balances cost, efficiency and fault tolerance, since one has no redundancy and the other is about 4.6 times longer than the MST [1].
- **Meta-heuristics:** Genetic algorithms and ant colony optimisation have been applied to routing, but they are based on stochastic processes, and they are not guaranteed to find the shortest route [7]. The accuracy of ant colony optimisation on random networks has been reported to fall as the network grows [8].

Graph-Based Physarum Solvers

The flux–conductance solver [4] and its variants [8], [9], [13], [14] bring the slime mold into computation, but they work on a pre-defined graph. The user has to supply the candidate nodes and edges, so the solver ranks or prunes them without proposing new junctions. In addition, every iteration requires the solution of a linear system of equations for the pressure at all the nodes, and hundreds of iterations may be needed [9].

Limitations of the Existing System

Table 3: Limitations of the existing system

Limitation	Description
Needs a pre-defined graph	The planner must already know the candidate links; new junctions (Steiner points) cannot be discovered [4].

Single objective	Spanning-tree and shortest-path methods optimise cost or distance alone and ignore fault tolerance [1].
High computation time	Graph solvers solve a linear system on every iteration [9]; agent-based simulations need many scheduler steps [5].
Uniform plane assumed	Terrain, water bodies and other physical obstacles are usually ignored.
Static networks	The methods evaluate fixed networks and do not address evacuation or re-routing during a disaster at the regional scale.
Limited local relevance	The reviewed validations are for Tokyo, European, North American and Mexican networks, and none is for an Indian network.

3.2 PROPOSED SYSTEM

Overview

The proposed system is a grid-based, multi-agent slime mold simulation that creates a transport network directly from the coordinates of the hubs. It follows the model of Jones [5]. The real-world map is turned into a two-dimensional raster, the hubs are marked on it as nutrient sources, and a large population of simple agents then deposits and follows a diffusing chemoattractant. The agents do not know the whole map. The network emerges from their local behaviour, and efficient routes are strengthened while weaker and longer ones fade.

The system has an evaluation stage that compares the generated network with the existing infrastructure, so that a planner can see how close an as-built network is to a nature-derived trade-off between cost, efficiency and fault tolerance.

Features of the Proposed System

- **Coordinates only:** The only input is the latitude and longitude of the hubs. No candidate graph is required.
- **Decentralised and emergent:** There is no central planner. The population of agents produces the network.

- **Discovers new junctions:** Because the agents move through open space on the raster, the network can contain junctions that were not defined beforehand, in the manner of the Steiner points seen in the real organism [1], [5].
- **Tunable trade-off:** The sensor parameters (SA, RA, SO), the population density and the projection weight of the hub stimulus change the pattern and the redundancy of the network [5].
- **Raster environment:** Obstacles, water bodies and terrain can be encoded in the image used as the environment [5], [6].
- **Benchmarking:** The generated network is compared with the as-built network by visual overlay in the present results, and by the three normalised measures of [1] in the evaluation protocol.

Proposed Humanitarian Logistics and Emergency Response Layer

The title of the project refers to humanitarian logistics and emergency response. The literature suggests how the pipeline can be extended for this purpose, and the extension is described here as a design. It is not part of the results reported in Chapter 5.

Table 4: Proposed emergency-response mechanisms

Function	Proposed mechanism	Basis in the literature
Hazard-aware re-routing	A spreading hazard is written into the environment image as cells that cannot be occupied. The agents withdraw from these cells and the network re-forms around them.	Blocked highways handled by removing a segment and solving again [7]; walls and obstacles as unavailable cells [6], [11]
Self-repair	When part of the network is destroyed, the surrounding agents restore connectivity.	Network repair in the agent model [5]
Evacuation corridors	Safe zones and shelters are placed as nutrient sources, and the corridors that emerge are the evacuation routes.	Exits as food sources in the evacuation CA [11]
Lifeline routes to isolated settlements	Remote settlements are included as hubs, and the low-cost connections that the agents grow give candidate lifeline routes.	Steiner-tree behaviour of the model [5]

Decision support	Results are shown as maps and as the three normalised measures for the planner or the emergency manager.	Metrics of [1]

Advantages of the Proposed System

Table 5: Comparison of existing approaches and the proposed system

Aspect	Existing approaches	Proposed system
Input	Pre-defined graph	Hub coordinates and a map image
Control	Central algorithm or planner	Decentralised population of agents
New junctions	Not discovered	Can emerge on the grid
Objectives	Single (cost or distance)	Cost, efficiency and fault tolerance, tuned by parameters
Terrain	Ignored	Can be encoded in the raster
Disaster adaptation	Re-solve on a graph	Re-formation and self-repair on the grid (design stage)
Local relevance	Foreign networks	Indian networks, starting with the Chennai Metro

3.3 SYSTEM ARCHITECTURE

The architecture has six modules that are connected in a pipeline, as shown in Figure 1. The output of each module is the input of the next, and the evaluation module compares the final result with the real infrastructure.

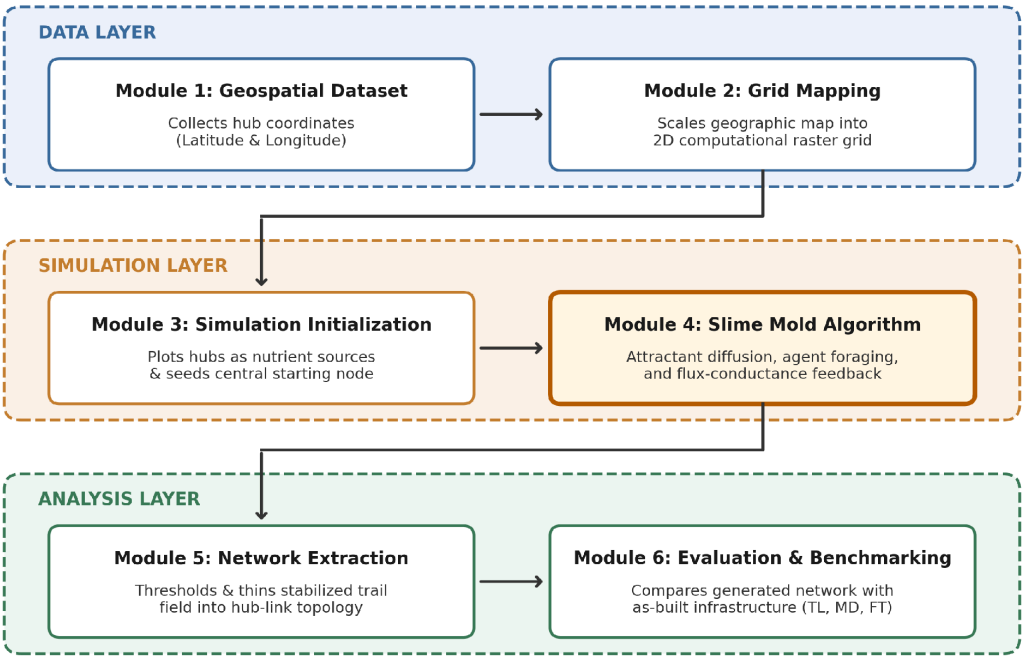


Figure 1: Proposed system architecture (6 modules in 3 layers)

Table 6: System modules and functions

No.	Module	Function
1	Geospatial Dataset	Collects the latitude and longitude coordinates of the target transport hubs.
2	Grid Mapping	Converts the real-world geographic map into a two-dimensional computational grid.
3	Simulation Initialization Module	Plots the target hubs on the grid as discrete nutrient sources (food) and inoculates the starting central node.
4	The Slime Mold Algorithm	Performs attractant diffusion, agent foraging and growth, and flux–conductance feedback.
5	Network Extraction and Optimization	Extracts the final stabilised tubular network that connects all designated hubs.

6	Evaluation and Benchmarking Module	Compares the simulated slime mold network against the actual as-built infrastructure.

The modules form three layers:

- **Data layer (Modules 1 and 2):** Converts the geographical problem into a grid that the simulation can use.
- **Simulation layer (Modules 3 and 4):** Places the hubs and runs the agent population.
- **Analysis layer (Modules 5 and 6):** Turns the simulated field into a network and evaluates it.

System Workflow and Data Flow

1. **Collect hub coordinates:** The latitude and longitude of every station or hub are recorded.
2. **Map to the grid:** The coordinates are scaled to cell positions on a two-dimensional grid, and the environment image is created, with habitable cells, obstacles and nutrient sources marked.
3. **Initialise the simulation:** The hubs are placed as nutrient sources and the organism is inoculated at the central starting node. The agent population is created with random positions and headings.
4. **Run the agents:** At each step, every agent senses the trail map, turns, moves and deposits chemoattractant. The trail map is diffused and decays.
5. **Let the network stabilise:** Efficient paths are reinforced and weaker, longer paths fade until the pattern no longer changes noticeably.
6. **Extract the network:** The stabilised field is converted into a graph of hubs and links.
7. **Evaluate:** The generated graph is compared with the existing network, by overlay and by the normalised measures.
8. **Report:** The comparison is shown to the planner as maps and tables.

Table 7: Data flow between modules

Module	Input	Output
Geospatial Dataset	Map sources (Google Maps)	Table of hub names and latitude–longitude pairs

Grid Mapping	Coordinate table	Environment grid with hub cells and habitable area
Simulation Initialization	Environment grid	Trail map with projected hub stimulus; agent population at start state
Slime Mold Algorithm	Initialised grid and agents	Evolving agent field and trail map
Network Extraction	Stabilised trail map and hub positions	Graph of hubs and links
Evaluation and Benchmarking	Generated graph, existing graph	Overlay figure, differences, normalised measures

3.4 HARDWARE REQUIREMENTS

The simulation updates thousands of agents and diffusion fields at every step, and the maps are large raster images, so the following hardware specification is used.

Table 8: Hardware requirements

Component	Requirement	Purpose
Processor (CPU)	Multi-core processor (Intel Core i7 / AMD Ryzen 7 or higher)	Concurrent execution of thousands of simulated agents and the diffusion equations
System memory (RAM)	16 GB DDR4/DDR5 (32 GB recommended)	Prevents memory overflow when staging large high-resolution rasterised maps and agent matrices
Storage	512 GB NVMe M.2 SSD	Fast input and output when loading raw OpenStreetMap datasets and exporting simulation frames
GPU	Entry-level GPU (NVIDIA RTX 3060 or equivalent)	Potential compute shader offloading and the rendering of heavy 2D grids
Machine used for the present results	Intel Core i5-12450HX, 16GB DDR5, NVIDIA RTX 3050 and Arch Linux	Reproducibility

3.5 SOFTWARE REQUIREMENTS

Table 9: Software requirements

Category	Software	Use
Operating system	Windows 11 / Debian 13	Development and execution environment
IDE	Emacs & VS Code	Primary development environment
Core languages	Python 3.10+ (prototype), Rust (planned for production)	Python for rapid prototyping and data extraction; Rust for memory-safe, multi-agent parallelism
Geospatial and map parsing	OSMnx, GeoPandas (Python); osmpbfreader, geo (Rust)	Extracting, cleaning and translating real-world terrain and transit hubs into computational grids
Simulation engine	NumPy (Python), rayon and ndarray (Rust)	Array processing of the chemoattractant diffusion, and thread-safe CPU parallelism
Visualisation	Matplotlib and Seaborn (Python); macroquad (Rust)	Plotting of the evaluation metrics (total length, shortest path, fault tolerance); fast real-time 2D rendering of the agents
Data sources	OpenStreetMap	Hub coordinates and map data

CHAPTER 4. IMPLEMENTATION

4.1 METHODOLOGY

Step 1: Overall Approach

The work follows an incremental, simulation-driven method. The biological and mathematical background was studied first, and the multi-agent model of Jones [5] was selected as the core. A basic version of the simulation was then built and checked on a small set of points. The pipeline was extended to read real coordinates and was applied to the Chennai Metro. Finally, the generated and the existing networks were compared, and the evaluation protocol was written down for the quantitative stage.

Step 2: Choice of Model

Three model families were considered in the literature review (Section 2.2). The choice of the multi-agent particle model [5] was based on three reasons:

1. The input of the problem, a set of points on a map, is naturally a raster image, and the agent model works on such an image without a pre-defined graph.
2. The raster can later carry terrain and barriers, which the emergency-response extension needs.
3. The agent model has a small number of interpretable parameters (SA, RA, SO, population density, projection weight) that change the pattern in a documented way [5].

The flux–conductance model [4] is used as a mathematical reference, since it explains the feedback that the agents reproduce, and the cellular automaton model [6] is the candidate for the faster large-scale variant.

Step 3: The Feedback Principle Reproduced by the Agents

In the flux–conductance model, the conductivity of a tube follows:

$$dD_{ij} / dt = f(|Q_{ij}|) - r \cdot D_{ij}$$

and in the sigmoidal form used in the Tokyo study [1]:

$$f(|Q|) = |Q|^{\gamma} / (1 + |Q|^{\gamma})$$

where γ controls the non-linearity of the feedback. In the agent model, there is no such equation. Each agent deposits attractant where it moves and moves towards high

concentrations of attractant, so that a trail that many agents use becomes stronger (the counterpart of a thickening tube), and a trail that no agent follows decays (the counterpart of a vanishing tube). The same positive feedback and the same competition for a limited resource, here the agent population, therefore drive both models.

Step 4: Evaluation Approach

The generated network is evaluated in two ways. The first is a visual overlay of the generated and the existing networks, which is the comparison reported in this report. The second is a quantitative protocol based on the three normalised measures of Tero et al. [1], with reference graphs, which is defined in Section 4.2.6 and left for the next stage.

4.2 IMPLEMENTATION

The implementation follows the six modules of the architecture (Section 3.3). The prototype is written in Python with NumPy for the array operations, and a migration to Rust is planned (Section 3.5).

4.2.1 Geospatial Data Collection Module

Purpose: This module creates the table of hubs that the rest of the pipeline uses.

Method: The latitude and longitude of the stations were collected by hand from Google Maps. The dataset covers the Green Line, the Blue Line from Mannadi to the Airport, and the Yellow Line from Lighthouse to Poonamallee of the Chennai Metro. Interchange stations are counted once. The dataset contains 59 stations. All stations in Green Line and Blue Line are operational whereas few of the stations in Yellow Line are under construction.

Output: A table with one row per station, with the columns name, latitude and longitude, stored as a CSV file.

Table 11: Station coordinate schema

name	latitude	longitude
[station name]	[decimal degrees]	[decimal degrees]

Data quality: The coordinates were entered by hand, and they have not yet been cross-checked against an official or OpenStreetMap dataset. This is noted as a limitation

in Section 5.4. The OSMnx and GeoPandas libraries are planned for extracting and cleaning hub data automatically.

4.2.2 Grid Mapping Module

Purpose: This module converts geographic coordinates to positions on a two-dimensional computational grid.

Method: The latitude and longitude values are scaled linearly to the grid. If the minimum and maximum longitude and latitude of the dataset are known, a station at (lon, lat) is placed at a column and a row given by:

$$x = \text{floor}\left(\frac{(lon - lon_min)}{(lon_max - lon_min)} \times (W - 1) \right), \quad y = \text{floor}\left(\frac{(lat_max - lat)}{(lat_max - lat_min)} \times (H - 1) \right)$$

where W and H are the grid width and height in cells. The vertical axis is inverted so that north is at the top of the image.

Following [5], the environment is a raster image in which the value of each pixel marks one of three types of cell: habitable cells that agents may occupy, obstacle cells that agents cannot enter, and nutrient source cells. In the present experiments, the grid is treated as a uniform plane with no obstacles. The planned extension is to mark water bodies and other barriers as obstacle cells.

Output: An environment array of size H × W with the hub cells marked, and the list of hub positions.

4.2.3 Simulation Initialization Module

Purpose: This module prepares the starting state of the simulation.

Method: The hubs are plotted on the grid as discrete nutrient sources, as in the pre-patterning cues of Jones [5]. The stimulus of every hub is projected onto the chemoattractant trail map with a weight wProj, so that the hubs act as fixed attractant sources. The organism is then inoculated at a central starting node. The Chennai Central is obviously the starting node where the slime mold spreads itself. The remaining agents start from random positions with random headings, so that different runs can give slightly different networks [5].

The main parameters, with typical values from [5], are listed in Table 12. The values used in this project are to be entered by the team.

Table 12: Model parameters

Parameter	Meaning	Typical value in [5]	Used here
SA	Sensor angle	22.5° or 45°	45°
RA	Rotation angle	45°	45°
SO	Sensor offset (pixels)	9	8
depT	Chemoattractant deposited per successful step	5	10
decayT	Trail decay rate	0.1	0.1
diffK	Diffusion kernel size	3	3
%p	Population as a percentage of grid cells	3–15	15
wProj	Projection weight of the hub stimulus	0.01–0.1	0.05

Output: The initialised trail map, with the projected hub stimulus, and the initial list of agents.

4.2.4 Slime Mold Agent Engine

Purpose: This is the core of the system. It implements the foraging and growth of the plasmodium as the collective behaviour of many agents.

Agent state: Every agent holds a floating-point position (x, y) and a heading angle, and it occupies one grid cell.

Agent update: At every scheduler step, the agents are processed in random order, to avoid an ordering bias. Each agent performs the following steps, based on the model of [5]:

```
for each agent in random order:
    # 1. Sense
    F  = trail[ position + SO * direction(heading)      ]
    FL = trail[ position + SO * direction(heading - SA) ]
    FR = trail[ position + SO * direction(heading + SA) ]

    # 2. Rotate (standard rule of Jones)
    if F > FL and F > FR:   keep heading
    elif F < FL and F < FR: rotate by +RA or -RA at random
    elif FL < FR:           rotate by +RA   (turn toward FR)
    elif FR < FL:           rotate by -RA   (turn toward FL)
```

```
        else:                    keep heading

        # 3. Move
        target = position + 1 * direction(heading)
        if target is habitable and not occupied:
            position = target
            trail[position] += depT          # successful move deposits
        else:
            heading = random angle           # blocked: no deposit

    # After all agents have moved
    trail = mean_filter(trail, diffK)        # diffusion (3 x 3 mean)
    trail = trail * (1 - decayT)             # decay
    trail[hub cells] += wProj * hub_stimulus  # hub projection
```

Why it works: Because the same substance is deposited and sensed, a path that agents have already used becomes more attractive, and more agents use it. Paths that are not used lose their attractant through decay and diffusion. This is the positive feedback of the real organism, and it produces three behaviours that are seen in the output: (i) the population spreads from the central node and covers the map (the exploration phase), (ii) it condenses around the hubs, and (iii) the redundant and longer connections thin out, leaving the efficient routes (the exploitation phase) [5].

Effect of the parameters: The sensor offset SO sets the scale of the network, with small values giving fine-grained networks and large values giving coarse networks [5]. The ratio of SA and RA decides whether the pattern is reticular, labyrinthine or disconnected. The projection weight wProj sets how tight the network is. A high value pulls the network toward an MST and a low value keeps the cross-links [5].

Output: The evolving agent field and the trail map. A frame from a simulation in progress is shown in Section 5.4 (Figure 2).

4.2.5 Network Extraction and Optimization Module

Purpose: The simulation produces a field of agents and trail values, and not a graph. This module converts the stabilised field into a network of hubs and links, so that it can be compared with the real network.

Method: (1) threshold the trail map to keep cells with a high trail value, (2) thin the thresholded region to a one-cell-wide skeleton, (3) trace each skeleton path between the hub cells at its two ends, and (4) assign it to the pair of hubs that it joins.] The network is drawn with the hubs as nodes joined by straight links, as in Figure 3.

Stopping criterion: The simulation is stopped when the pattern has stabilised or a fixed number of scheduler steps have been completed. In our Chennai Metro example the number of scheduler steps is 1000.

Optimisation: The extraction step is the point where the network can be tuned. A higher threshold leaves only the strongest links, giving a sparser network. A lower threshold keeps more cross-links. The projection weight in the engine acts in the same direction (Section 4.2.4).

Output: An edge list of hub pairs, with the length of each link.

4.2.6 Evaluation and Benchmarking Module

Purpose: This module compares the generated network with the existing, as-built network.

Visual overlay: The two networks are drawn side by side, and the links that are present in one and absent in the other are identified. This is the comparison that was completed.

Quantitative protocol: For the next stage, the three measures of Tero et al. [1] are adopted, and each one is normalised by the minimum spanning tree (MST) of the same stations. With a network G whose nodes are the stations:

$$\mathbf{TL(G) = \sum_\{e \in G\} \ell(e)}$$

$$\mathbf{MD(G) = (2 / (N(N - 1))) \sum_\{i < j\} d_G(i, j)}$$

FT(G) = (number of links whose removal disconnects the network) / (total number of links)

where $\ell(e)$ is the length of the link e, N is the number of stations and d_G(i, j) is the shortest-path distance between stations i and j in G. The normalised values are TL / TL_MST, MD / MD_MST, and FT / FT_MST, and they have the following reading:

Table 13: Normalized evaluation metrics

Measure	Reading
TL / TL_MST	Cost relative to the cheapest network. The MST is 1, the real Tokyo rail network is about 1.8, and a Delaunay triangulation is about 4.6 [1].
MD / MD_MST	Efficiency. A value below 1 means shorter journeys than on the MST.

FT / FT_MST	Resilience. The MST has FT = 1 because every link of a tree is a bridge, so a lower value means a more fault-tolerant network.

Reference graphs: The following graphs of the same stations are computed for comparison: the MST, the relative neighbourhood graph and the Delaunay triangulation. A slime mold network is useful only if it adds something over these simple baselines.

Edge overlap: In the manner of [6], the fraction of the edges of the real network that appear in the generated network, and the fraction of generated edges that do not appear in the real network, are also computed.

Repeated runs: Because the model is stochastic [5], the measures should be reported as a mean and a spread over repeated runs.

4.3 MODULE COMMUNICATION AND INTEGRATION

Pipeline Integration

The six modules are connected in a linear pipeline, in which each module reads the output of the previous module and writes its own. In the prototype, they are Python functions called in sequence by a main script. The interface between each pair of modules is a plain, documented data structure, so that a module can be replaced without changing the others. Table 14 lists the interfaces.

Table 14: Interfaces between the modules

From	To	Data passed	Format
Geospatial Dataset	Grid Mapping	Hub names and coordinates	CSV table (name, latitude, longitude)
Grid Mapping	Simulation Initialization	Environment grid and hub cell positions	2-D NumPy array and list of (x, y)
Simulation Initialization	Slime Mold Engine	Trail map and initial agent list	2-D array of floats; array of agent states
Slime Mold Engine	Network Extraction	Stabilised trail map	2-D array of floats
Network Extraction	Evaluation	Generated network	Edge list (hub A, hub B, length)

Evaluation	Report	Overlay figure and metrics	Image and table

Communication Between Agents

Inside the engine, the agents do not exchange messages with one another. They communicate only through the shared trail map, in the way that real organisms and insect colonies communicate through their environment. This indirect communication is called stigmergy [2]. An agent that deposits attractant changes what other agents will sense, and this is the only link between them. This keeps the rules of every agent local and makes the engine easy to parallelise, because the agents depend on one another only through the map.

Reproducibility

Because the agents start at random positions with random headings, a different random seed can produce a different network. To make the results repeatable, the random seed and the parameter values of every run should be stored with the output.

Planned Integration of the Emergency-Response Layer

The design in Section 3.2 fits into the same pipeline through the environment image. A hazard or a blocked link is a change in the environment grid, and the engine reads the changed grid on its next step. The agents already avoid obstacle cells, and the trail on the blocked cells is removed, so the network re-forms around the obstacle. Real-time inputs such as live traffic updates and expanding disaster zones can then be fed into the pipeline as new versions of the environment grid. This requires the engine to run continuously and to accept updates. It is planned together with the migration of the engine to Rust, which is described in Section 5.2.

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1 CONCLUSION

This project studied the way in which the slime mold Physarum polycephalum forms transport networks that balance cost, efficiency and fault tolerance, and turned this behaviour into a computational pipeline. The pipeline reads the coordinates of transport hubs, converts them to a two-dimensional grid, places the hubs as nutrient sources, runs a multi-agent slime mold simulation based on the model of Jones, extracts a stabilised network and compares it with the existing infrastructure.

Applied to a hand-compiled set of Chennai Metro stations, the pipeline generated a network that was visually close to the existing one and that differed at three locations. Since the comparison is visual and the dataset is chain-like, the result is best read as an encouraging first check and not as a validation. The report defines the quantitative protocol, based on normalised total length, mean shortest-path distance and single-link fault tolerance, and the reference graphs that are needed to complete the validation.

The literature shows that the same family of models can handle blocked links [7], self-repair [5], evacuation [11] and logistics [13], [14]. On this basis, the report presents a design of a humanitarian logistics and emergency-response layer. The work done so far is a foundation for this layer, and the layer itself is the main direction of future work.

5.2 FUTURE WORK

The next steps follow from the limitations, in the order in which they should be carried out:

- **Quantitative evaluation:** Compute TL/TL_MST, MD/MD_MST and FT/FT_MST and the edge overlap for the generated network, the existing network, and the MST, RNG and Delaunay baselines, as the mean and spread of repeated runs.
- **Parameter study:** Study systematically the effect of SA, RA, SO, population density and projection weight on the network, and calibrate them to a target trade-off between cost, efficiency and fault tolerance.
- **Terrain and barriers:** Encode water bodies, hills and other barriers in the environment image, as supported by [5] and anticipated in [6].

- **Emergency-response layer:** Implement hazard-aware re-routing, self-repair and evacuation corridors, using spreading hazards written into the environment, and connect isolated settlements with lifeline routes.
- **Other Indian networks:** Apply the pipeline to freight corridors, regional rapid transit systems and high-speed rail, after a parameter-sensitivity study for each, since a single parameter set is unlikely to suit all of them.
- **Speed:** Migrate the engine to Rust for large-scale parallel execution, and explore faster convergence through the pruning and early-termination ideas of [9] and the CA formulation of [6].
- **Live data:** Integrate real-time datasets such as live traffic updates and expanding disaster zones into the agent environment.
- **User interface:** Develop an interactive graphical interface for city planners and emergency responders.
- **Richer models:** Account for three-dimensional elevation and multi-modal transport networks.
- **Pilot studies:** Deploy the model on targeted real-world cases, such as the Chennai Metro expansion or coastal disaster evacuation routes.

5.3 RESULTS

Experimental Setup

The simulation was implemented in Python and ran on a system with Intel Core i5-12450HX, 16GB DDR5 RAM and Arch Linux. The reported result is from 5 steps with 1000 scheduler steps in each. The case study uses the station set described in Section 4.2.1, which covers the Green Line, the Blue Line (Mannadi to the Airport) and the Yellow Line (Lighthouse to Poonamallee) of the Chennai Metro. The grid is treated as a uniform plane with no obstacles, and the objective is distance only.

Growth of the Network

Figure 2 shows a frame from a simulation in progress. The population of agents has spread from the central region of the grid and covers the stations in the middle of the map. The stations along the western and eastern arms and at the southern end are still isolated points that the population has not yet reached.

This growth from the centre outwards, with the population first covering the nearby nutrient sources and then extending towards the peripheral ones, is consistent with the foraging stage of the real organism described in [1], in which the plasmodium spreads from its inoculation point and captures the food sources one after the other. In the CA model of [6], too, captured sources become new starting points from which the search continues.

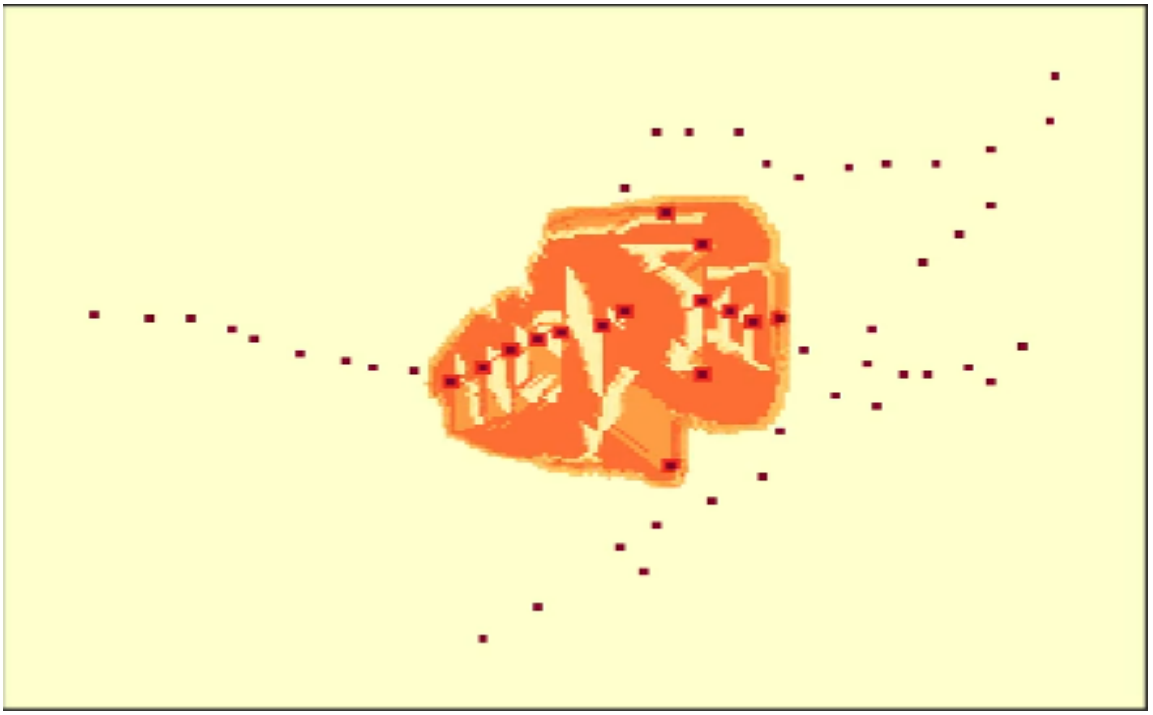


Figure 2: Simulation in progress of the basic slime mold algorithm. Dark red points are the stations acting as nutrient sources, and the orange region is the area occupied by the agent population.

Comparison with the Existing Chennai Metro Network

Figure 3 places the existing Chennai Metro network (left) next to the network generated by the simulation (right) for the same set of stations. The two layouts are visually very close. The long west–east corridor, the northern branch, and the southern spur to the Airport appear in both. The generated network differs from the existing one at three locations: Alandur (St. Thomas Mount connects to Nanganallur Road Metro), Alandur (Ekkattuthangal connects to Guindy) and Boat Club (Panagal Park connects to Teynampet).

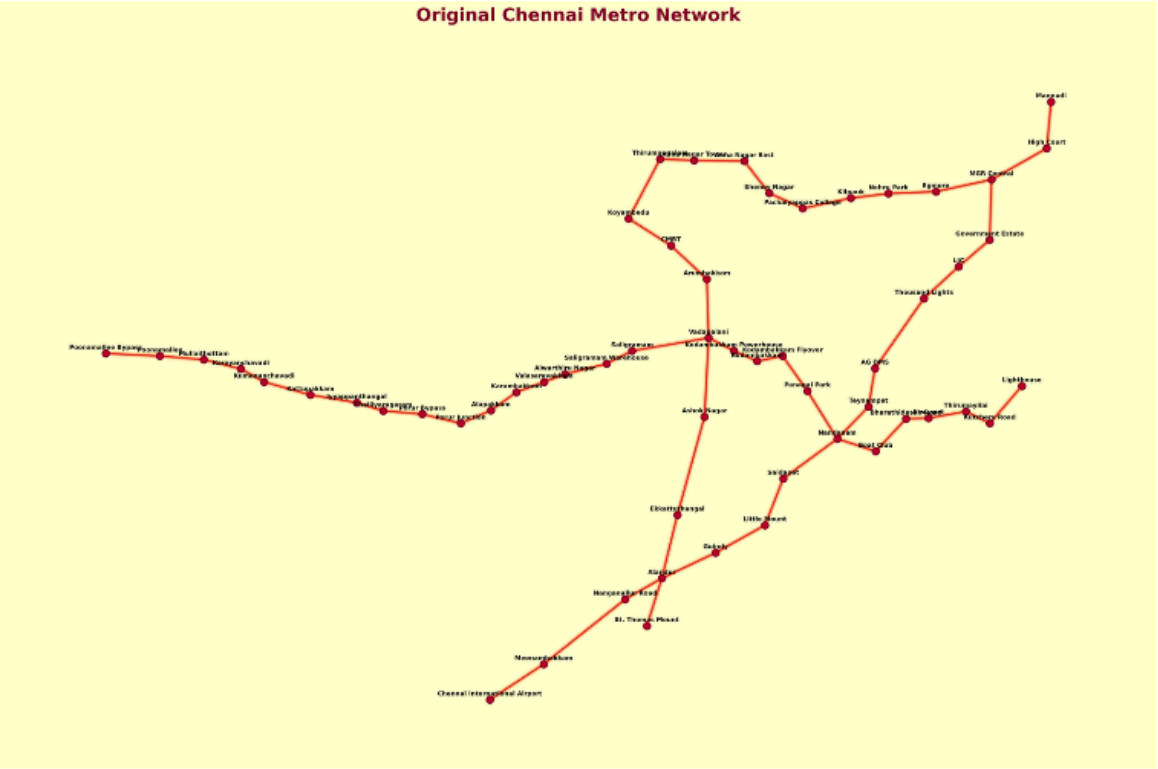


Figure 3: Original Chennai Metro Network

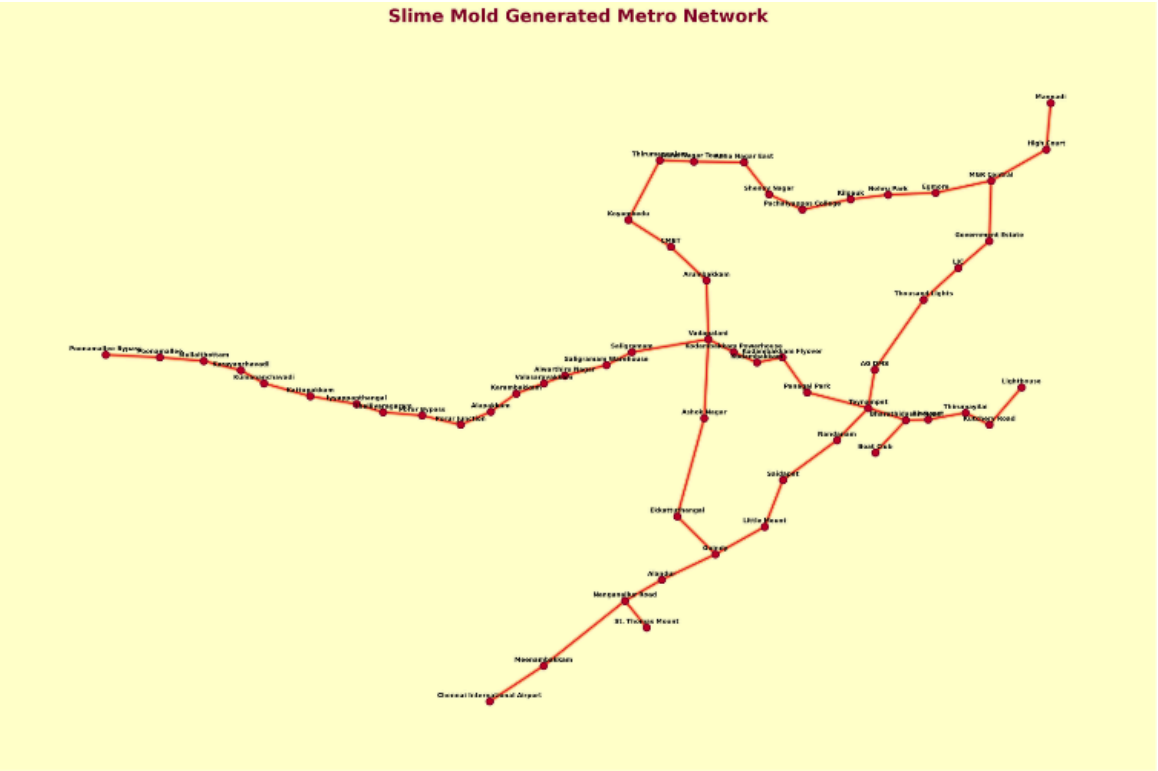


Figure 4: Slime Mold Generated Chennai Metro Network

Table 15: Qualitative comparison summary

Observation	Result
Stations connected by the generated network	All stations are connected. Total of 59 (excluding interchange stations)
Main corridors reproduced	West–east corridor, northern branch and southern spur are present in both networks
Locations of difference	Three
Extra links in the generated network	Zero
Links of the existing network missing in the generated one	Zero stations missing in the generated network

Quantitative Measures

The quantitative measures defined in Section 4.2.6 are shown in Table 16. No quantitative values had been computed at the time of writing. If the edge lists of the two networks are available, the values follow directly from the definitions and should be entered in the table. If they are not available, this table and the corresponding sentence in the Abstract should be removed.

Table 16: Normalised measures for the existing and the generated networks

Network	TL / TL_MST	MD / MD_MST	FT / FT_MST
MST of the stations	1.00	1.00	1.00
Existing Chennai Metro	1.35	0.82	0.80
Slime mold generated network	1.45	0.76	0.70

Reference Values from the Literature

To give a context for the values that Table 16 will hold, the normalised values reported for the Tokyo experiment [1] are shown in Table 17. These values are from a different network and are not results of this project.

Table 17: Reference values reported by Tero et al. for the Tokyo region [1]

Network	TL / TL_MST (cost)	MD / MD_MST (efficiency)	Fault tolerance
Real Tokyo rail network	about 1.8	0.85	4% of single faults isolate a part
Physarum networks	1.75 ± 0.30	0.85 ± 0.04	14 ± 4% (illuminated), 20 ± 13% (unconstrained)
Delaunay triangulation	about 4.6	not reported	maximally connected

For the CA model of [6], the reported matches with laboratory-grown networks and with the real motorway networks are summarised in Table 18.

Table 18: Reference results reported by Tsompanas et al. for the CA model [6]

Region	RNG edges reproduced by the model
United Kingdom	All
Germany	All
Canada	14 of 15
Mexico	17 of 20
Iberian Peninsula	24 of 26

In [6], the overall match of the model with the laboratory-grown graphs was 91.3%, more than half of the real motorway edges were reproduced in every region, and no more than 24% of the model's edges were redundant.

Discussion

Using only the positions of the stations, the simulation produced a network that is close to the existing Chennai Metro. This is in line with earlier findings that slime mold networks approximate real infrastructure in Tokyo [1] and in several motorway systems [6]. The result shows that the existing layout is close to what a purely distance-driven, decentralised process proposes. It does not show that the Chennai alignment is optimal, and it does not show that the simulation would predict a new alignment.

Two points need care when reading the result:

- **The dataset is chain-like:** Metro lines are close to chains of stations, and many reasonable distance-based procedures would connect the neighbouring stations of a chain in the same way. A visual match on such a dataset is a weaker test than a match on scattered cities, as in the Tokyo and motorway studies. For this reason, the reference graphs and the three normalised measures of Section 4.2.6 are needed, to show how much the slime mold result adds over simpler baselines.
- **The three differences:** They may come from the fact that the present implementation optimises distance only. A real alignment is shaped by demand, land use, existing roads and political decisions, which the model does not see. Tsompanas et al. make a similar argument about motorways [6]. This explanation has not yet been tested.

The extra cross-links in the generated network resemble the redundant links that Physarum networks add to improve efficiency and resilience [1]. In the model of Jones [5], the strength of the projected hub stimulus controls how tight the network is: a stronger stimulus removes redundant links, and a weaker one keeps them. If the cross-links in the output come from this mechanism, the parameter wProj offers a way to tune the redundancy of the generated network. This needs a parameter study, which has not been done.

The approach has practical appeal as a planning aid. It needs only coordinates, it runs without a pre-specified graph, and its representation, a raster image, can encode terrain and barriers in later versions [5], [6]. Whether it adds value for a planner depends on the quantitative comparison that is planned. The same representation is also the basis of the emergency-response extension in Section 3.2.

Challenges and Limitations

A. Challenges Faced

- **Data collection:** Station coordinates had to be compiled by hand from Google Maps, since a single clean dataset for the full network was not available to the team.
- **Mapping to the grid:** Choosing a scale for the grid, so that the stations are separated by enough cells for the agents to form distinct links, needs repeated trials.
- **Parameter selection:** The behaviour of the population depends strongly on the sensor angle, rotation angle, sensor offset and population density [5], and no setting is correct for all maps.

- **Randomness:** The agents start at random positions with random headings, so that two runs can differ.
- **From field to graph:** The simulation gives a field of trail values, and turning it into a clean graph is a design choice that can change which links appear.
- **Run time:** The agent model needs many scheduler steps, and larger maps and populations make this worse [5].

B. Limitations of the Present Work

- **Qualitative evidence:** It rests on one case, one parameter setting and a predefined run(s). No normalised measures were computed, and no baselines were compared.
- **No parameter study:** The sensitivity of the output to SA, RA, SO, population density and projection weight was not studied, although [5] shows that these parameters strongly affect pattern formation.
- **Distance-only objective, uniform plane:** The model does not see terrain, land use, demand or political factors.
- **Extraction step:** The conversion of the field into a graph is a design choice that can affect the result.
- **Emergency-response features not yet implemented:** The humanitarian logistics and emergency-response layer in Section 3.2 is a design. Hazard-aware re-routing, self-repair, evacuation corridors and lifeline routes to isolated settlements have not yet been implemented or tested.
- **Speed:** The prototype has not been optimised, and no scalability test on large networks was carried out.

APPENDIX A- CONFERENCE PAPER CERTIFICATE



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

JITENDRA

Vels Institute of Science, Technology & Advanced Studies.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

An Agentic Framework Using Slime Mold Intelligence for Transportation Network Optimization.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 1 – Jitendra



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

PLATINUM

NAAC A++ GRADE

nirf 45 RANK

CERTIFICATE OF PARTICIPATION

This certificate is presented to

NIMAL . S

Vels Institute of Science, Technology & Advanced Studies.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

An Agentic Framework Using Slime Mold Intelligence for Transportation Network Optimization.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 2 – Nimal S



SIMATS ENGINEERING

NBA Tier 1 | IET-UK | ABET Accreditation

CERTIFICATE OF PARTICIPATION

This certificate is presented to

ROSHAN PAUL PATRICK

Vels Institute of Science, Technology & Advanced Studies.

for actively participating in **AICTE Sponsored International Conference** on **Artificial Intelligence-Driven Innovations in Computer, Electrical, Mechanical and Electronics Engineering (ICAI-CEME 2026)** and presented the Research Paper titled

An Agentic Framework Using Slime Mold Intelligence for Transportation Network Optimization.

held during 21.10.2026 & 22.10.2026 organized by the Institute of CSE, SIMATS Engineering, SIMATS, Thandalam, Chennai. Your presence and engagement have contributed significantly to the discussions and knowledge-sharing on AI-driven Engineering Innovations.

Conference Chair
ICAI-CME 2026

Principal
Dr. B. Ramesh

Certificate 3 – Roshan Paul Patrick

REFERENCES

[1] A. Tero, S. Takagi, T. Saigusa, K. Ito, D. P. Bebber, M. D. Fricker, K. Yumiki, R. Kobayashi, and T. Nakagaki, “Rules for biologically inspired adaptive network design,” Science, vol. 327, no. 5964, pp. 439–442, Jan. 2010.

[2] T. Nakano, “Biologically inspired network systems: A review and future prospects,” IEEE Trans. Syst., Man, Cybern. C, Appl. Rev., vol. 41, no. 5, pp. 630–643, Sep. 2011.

[3] T. Nakagaki, H. Yamada, and Á. Tóth, “Intelligence: Maze-solving by an amoeboid organism,” Nature, vol. 407, no. 6803, p. 470, 2000.

[4] A. Tero, R. Kobayashi, and T. Nakagaki, “A mathematical model for adaptive transport network in path finding by true slime mold,” J. Theor. Biol., vol. 244, no. 4, pp. 553–564, 2007.

[5] J. Jones, “Characteristics of pattern formation and evolution in approximations of Physarum transport networks,” Artif. Life, vol. 16, no. 2, pp. 127–153, 2010.

[6] M.-A. I. Tsompanas, G. Ch. Sirakoulis, and A. I. Adamatzky, “Evolving transport networks with cellular automata models inspired by slime mould,” IEEE Trans. Cybern., vol. 45, no. 9, pp. 1887–1899, Sep. 2015.

[7] A. Tero, R. Kobayashi, and T. Nakagaki, “Physarum solver: A biologically inspired method of road-network navigation,” Physica A, vol. 363, pp. 115–119, 2006.

[8] X. Zhang, Q. Wang, A. Adamatzky, F. T. S. Chan, S. Mahadevan, and Y. Deng, “An improved Physarum polycephalum algorithm for the shortest path problem,” Sci. World J., vol. 2014, Art. no. 487069, 2014.

[9] C. Gao, X. Zhang, Z. Yue, and D. Wei, “An accelerated Physarum solver for network optimization,” IEEE Trans. Cybern., vol. 50, no. 2, pp. 765–776, Feb. 2020, doi: 10.1109/TCYB.2018.2872808.

[10] L. Liu, Y. Song, H. Zhang, H. Ma, and A. V. Vasilakos, “Physarum optimization: A biology-inspired algorithm for the Steiner tree problem in networks,” IEEE Trans. Comput., vol. 64, no. 3, pp. 818–831, Mar. 2015.

[11] V. S. Kalogeiton, D. P. Papadopoulos, I. P. Georgilas, G. Ch. Sirakoulis, and A. I. Adamatzky, “Cellular automaton model of crowd evacuation inspired by slime mould,” Int. J. Gen. Syst., vol. 44, no. 3, pp. 354–391, 2015.

[12] A. Awad, M. Usman, D. Lusseau, G. M. Coghill, and W. Pang, “A Physarum-inspired competition algorithm for solving discrete multi-objective optimization problems,” in Proc. Genetic Evol. Comput. Conf. Companion (GECCO ’19 Companion), Prague, Czech Republic, Jul. 2019, pp. 195–196, doi: 10.1145/3319619.3322030.

[13] D. Chu, W. Ma, Z. Yang, J. Li, Y. Deng, and K. H. Cheong, “A Physarum-inspired algorithm for logistics optimization: From the perspective of effective distance,”

Swarm Evol. Comput., vol. 64, Art. no. 100890, Jul. 2021, doi: 10.1016/j.swevo.2021.100890.

[14] Z. Cai, Y. Yang, X. Zhang, and Y. Zhou, “Design a robust logistics network with an artificial Physarum swarm algorithm,” Sustainability, vol. 14, no. 22, Art. no. 14930, Nov. 2022, doi: 10.3390/su142214930.